ComputerLM

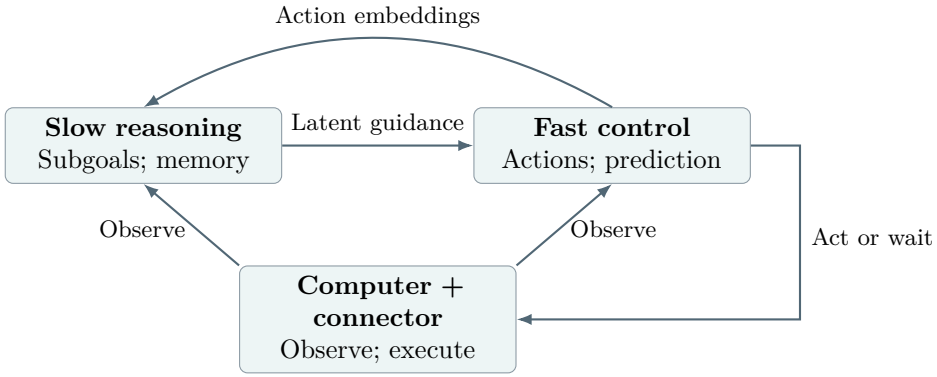
Persistent State and Asynchronous Reasoning for Fast Computer Use

J.D. Krasnick github.com/JDKrasnick/computerlm

Summary. ComputerLM combines a small recurrent controller with a deeper reasoning path that continues while the controller acts. Both observe the interface and maintain living task state. Slow reasoning communicates subgoals through learned latent vectors; fast control grounds them into actions and predicts short-term consequences. The objective is to match the strongest available GUI systems on easy and medium browser/desktop workflows, prioritizing faster completion and then lower cost. The integrated model remains untrained; these gains are hypotheses to test.

1 Proposed architecture

Many workflows contain a planning decision followed by routine interactions. ComputerLM aims to reuse that decision, carry progress forward, and overlap useful reasoning with execution and application delays.



Fast path. A custom recurrent controller combines fresh visual features, action history, persistent state, and slow guidance. Only this path executes human-style GUI actions: clicking, typing, scrolling, dragging, keyboard shortcuts, or waits. The initial interface excludes shell, direct-file, DOM, and API shortcuts. Fast control predicts bounded action consequences; actual observations establish their effects.

Slow path. Adapt a suitably sized pretrained VLM into an input encoder, shared recurrent middle blocks, and guidance readout. It reasons continuously during an active task, including while fast control acts or waits. Guidance expresses dependencies such as *find X to obtain Y* in latent space. A trained interface makes those vectors useful to the controller; slow reasoning cannot directly act on the computer. The checkpoint and size remain open.

Perception and state. Begin with native VLM screen encoding plus available accessibility/UI text. Cache identical encoder inputs under matching settings; fully encode changed screens. Recurrent state survives interactions and pause/resume, then resets between unrelated tasks. Preserve exact instructions, text payloads, action acknowledgements, and evidence references alongside compressed state. Action embeddings report operations and arguments; they do not represent every possible environment state.

2 Design rationale and development

Recurrent Memory Transformer carries learned memory between input segments [1]. Fast-in-Slow and DuoCore-FS provide robotics precedents for fast/slow cooperation and asynchronous latent guidance [2, 3]. In GUI research, iSHIFT conditionally requests more visual processing after latent deliberation [4]; GUI-JEPA studies lightweight latent action scoring [5]. These support components, not the complete architecture. Native VLM perception has a detailed precedent in Qwen2.5-VL [6].

Fresh evidence. Screens become available to both paths immediately. Compare next-cycle intake with trained early/middle/late adapters, preserving state and measuring reaction delay and recovery. Initial episodes follow one instruction; later live steering can revise goal state through the slow update interface and propagate revised guidance.

Adaptive execution. A central ablation compares immediate highest-scoring action selection with learned deferral for additional fast computation or fresher slow guidance. Prediction supports this decision; a prediction-disabled control tests its separate contribution. Both variants retain continuous slow reasoning and ordinary waits for pending UI operations. Action probability alone is not assumed to measure correctness.

Local constraints. Prefer Apple-Silicon-class inference with compact models, bounded state, and realistic unified-memory use. Initially target one GPU with CPU support. Custom scheduling may be needed; profile fit, slow progress, and fast tail latency. Exact hardware and serving software remain open. Approximate visual reuse is optional future work.

2.1 Staged training proposal

Training hardware and compute/data budgets remain unspecified. Begin with a small diagnostic:

1. **Preserve pretrained capability.** Establish one-pass parity and stable recurrent adaptation. Initially retain native perception; select frozen weights, adapters, and recurrent depth from development evidence.

2. **Learn execution and guidance together.** Train the controller and latent interface on chronological screen–action–outcome sequences. Include persistent memory, exact text access, failed actions, recovery, and delayed or missing guidance.

3. **Add prediction and adaptive computation.** Supervise fast predictions with actual transitions; train and compare action/deferral policies. Collect learner failures for refinement, considering RL after useful supervised behaviour is established.

Training uses only information available at each decision. Keep held-out outputs and corrections outside training. Memory across observations and repeated internal reasoning depth are separate: the latter adds computation even when no thinking text is generated.

3 Evaluation objective

Use a balanced browser/desktop suite with fixed easy/medium criteria. Select the strongest available GUI references before testing; retain same-backbone sequential and component controls. Match observations, GUI actions, and resets. Equalize hardware for local controls; disclose hosted-reference hardware and network differences. Measure complete trajectories rather than action latency alone [7].

4 Evidence required and remaining decisions

Primary result: faster completion at matched accuracy. Predeclare the acceptable accuracy difference, task mix, repetitions, and improvement thresholds before held-out testing. Verify exact task outcomes independently and report failed attempts. Include timing on jointly solved tasks without dropping other attempts from the main results.

Secondary result: lower inference cost. Count both paths, perception, prediction, state updates, synchronization, verification, and recovery. Report task-time distributions, tail latency, memory, and compute separately. For local inference, measure energy where possible and disclose assumptions behind monetary estimates. A faster system can still consume more computation; claim joint speed/cost improvement only when both are measured.

Qualitative grading follows the repository contract: ChatGPT through the signed-in Codex CLI, after each episode. Judge latency is separate, judge failure stops the batch, and favourable qualitative scores cannot override a failed exact outcome check.

Open decisions. Text-composition responsibility and completion/stall reporting remain behaviour choices. Development experiments address:

- **Intake and memory:** cycle length, adapter location, state capacity, and exact-record access.
- **Learned interfaces:** guidance capacity, fusion, publication/compatibility checks, and whether action embeddings retain all required arguments.
- **Fast decisions:** controller capacity, prediction targets/horizons, and when deferral earns its time and compute.
- **Training and runtime:** data budget, trainable components, recurrent training windows, local hardware fit, and scheduling policy.

Contribution and limits. The contribution sought is evidence that this integration improves practical computer-use speed while preserving success, with cost measured independently. The main risks are stale guidance, lost constraints, incorrect predictions, and continuous slow computation erasing the savings. Detailed alternatives and decision records remain in `paper/design-questions.md`.

References

[1] A. Bulatov, Y. Kuratov, and M. S. Burtsev. *Recurrent Memory Transformer*. NeurIPS, 2022. arXiv:2207.06881.

[2] H. Chen et al. *Fast-in-Slow: A Dual-System Foundation Model Unifying Fast Manipulation within Slow Reasoning*. 2025. arXiv:2506.01953.

[3] Astribot Team. *Asynchronous Fast-Slow Vision-Language-Action Policies for Whole-Body Robotic Manipulation*. 2025. arXiv:2512.20188.

[4] S. Mehrotra et al. *iSHIFT: Lightweight Slow-Fast GUI Agent with Adaptive Perception*. CVPR, 2026. arXiv:2512.22009.

[5] B. Wang et al. *GUI-JEPA: Candidate-Conditioned JEPA Latents as a Fast Action Policy for Mobile GUI Agents*. Research report, 2026. arXiv:2610.36877.

[6] Qwen Team. *Qwen2.5-VL Technical Report*. 2025. arXiv:2502.13923.

[7] P. Aggarwal et al. *cua-speedrun: Standardized Benchmarking of the Speed of Computer-Use Agents*. 2026. arXiv:2609.40284.